K.R. Mangalam University

School of Engineering & Technology

Assignment

Unit 1 – Foundations & Algorithmic Analysis
Data Structures (ETCCDS202)

Student Name : Sanchit
Roll Number : 2501730475
Section : A
Program : B.Tech AIML
Course Code : ETCCDS202
Submission Date : 05 / 03 / 2026

Submitted To : Mrs. Neetu Chauhan

School of Engineering & Technology
K.R. Mangalam University, Gurugram

INDEX

Sr. No	Topic	Page No.
1	Stack ADT	1
2	Recursive Factorial	2
3	Recursive Fibonacci	3
4	Tower of Hanoi	4
5	Recursive Binary Search	5
6	Paradigm Reflection	6

1.Stack ADT — Concept and Implementation

1.1 What is an Abstract Data Type (ADT)?

An **Abstract Data Type (ADT)** is a mathematical model for a data structure that defines *what operations* can be performed and what behavior is expected — without specifying *how* those operations are implemented internally. Think of an ADT as a contract or interface: the user only knows the available operations and their expected results, not the underlying code.

For example, a **Stack ADT** defines operations like push, pop, peek, isempty, and size. It does not matter whether the internal storage is a Python list, a linked list, or an array — the behavior remains the same. This separation of interface from implementation is the core idea of an ADT.

1.2 Why are Stack Operations O(1)?

All five core Stack operations run in **O(1) — constant time**:

- **push()**: Appending to the end of a Python list is O(1) amortized because Python over-allocates memory in chunks.
- **pop()**: Removing the last element of a Python list is O(1) — no shifting needed.
- **peek()**: Reads the last element by index — O(1).
- **isempty()**: Checks if length is zero — O(1).
- **size()**: Python maintains an internal count for lists — O(1).

Since no element needs to be moved or searched, the time does not grow with the number of elements, making Stack highly efficient.

1.3 Use of StackADT in This Toolkit

The StackADT is actively used in two places: (1) **Tower of Hanoi** — every move instruction is pushed onto hanoi_moves_stack so the complete move sequence is stored and can be reviewed. (2) **Binary Search** — every mid-index inspected during the search is pushed onto mid-stack, making the divide-and-conquer process traceable step-by-step.

2. Recursive Factorial — Complexity Analysis

2.1 How the Recursion Works

The factorial function is defined as **n! = n x (n-1) x ... x 1** with base case **0! = 1**. The recursive implementation makes one call per integer from n down to 0

2.2 Complexity Table

Case	Time Complexity	Space Complexity
Best Case	O(n)	O(n)
Average Case	O(n)	O(n)
Worst Case	O(n)	O(n)

- **Time Complexity O(n)**: There are exactly n+1 recursive calls. Each call does O(1) work (one multiplication). Total = O(n).
- **Space Complexity O(n)**: Each recursive call occupies one stack frame. At most n+1 frames exist simultaneously before the base case is reached.
- **Big-Omega O(n)**: We always make exactly n+1 calls regardless of input, so the lower bound is also O(n).
- **Big-Theta Theta(n)**: Since both upper and lower bounds are n, factorial is **Theta(n)** is tight bound .

3. Recursive Fibonacci — Naive vs Memoized

3.1 Why Naive Fibonacci is Slow

The naive formula is **fib(n) = fib(n-1) + fib(n-2)**. The problem is that the same sub-problems are recomputed many times:

For n=30, the naive version makes over **2.6 million calls**, while the memoized version makes only **59 calls**. The exponential blowup happens because every fib(k) for small k is recalculated repeatedly across different branches of the recursion tree.

3.2 Complexity Table

Case	Time Complexity	Space Complexity
Naive — All Cases	$O(2^n)$	O(n)
Memoized — All Cases	O(n)	O(n)

- **Naive Time $O(2^n)$**: Each call spawns two more calls. The recursion tree has approximately 2^n nodes. Total work is exponential.
- **Naive Space O(n)**: Despite exponential calls, the call stack depth is at most n (the longest path from root to a leaf in the recursion tree).
- **Memoized Time O(n)**: Each unique value fib(0) through fib(n) is computed exactly once and cached. Subsequent lookups cost O(1). Total unique subproblems = n+1, so time = O(n).
- **Memoized Space O(n)**: The cache dictionary stores n+1 values, and the call stack is at most n deep.

3.3 Call Count Evidence

Case	Time Complexity	Space Complexity
n = 5	15 calls	9 calls
n = 10	177 calls	19 calls
n = 20	21,891 calls	39 calls
n = 30	2,692,537 calls	59 calls

4. Tower of Hanoi — Recursion and Complexity

4.1 How the Recursion Works

To move **n disks** from source A to destination C using auxiliary B:

1. Move the top n-1 disks from A to B (using C as auxiliary).
2. Move the largest disk (disk n) from A to C directly.
3. Move the n-1 disks from B to C (using A as auxiliary).

Base case: When n = 1, move disk 1 directly from source to destination.

4.2 Why 2^n - 1 Moves?

Let T(n) = number of moves for n disks:

Solving: T(n) = 2T(n-1)+1 = 2(2T(n-2)+1)+1 = ... = $\mathbf{2^n - 1}$. For n=3: 2^3 - 1 = 7 moves (verified in output).

4.3 Complexity Table

Case	Time Complexity	Space Complexity
All Cases	$O(2^n)$	O(n)

- **Time $O(2^n)$**: Exactly 2^n-1 move operations are performed, each O(1). Total = $O(2^n)$.
- **Space O(n)**: Recursion depth equals n. No additional storage is used beyond the call stack, so space = O(n).

5. Recursive Binary Search — Divide and Conquer

5.1 How the Algorithm Works

Binary search works on a **sorted array**. At each step it computes the middle index and compares arr[mid] with the target key:

- If arr[mid] == key, return mid (found).
- If arr[mid] < key, search the right half (low = mid+1).
- If arr[mid] > key, search the left half (high = mid-1).

Base case: When low > high, search space is exhausted — return -1.

5.2 Recurrence and Complexity

Each recursive call halves the problem size. Each level does O(1) work. After $\log_2(n)$ halvings the array is empty. Result: **T(n) = O(log n)**.

5.3 Cases Explained

Case	Time Complexity	Space Complexity
Best Case	O(1)	O(log n)
Average Case	O(log n)	O(log n)
Worst Case	O(log n)	O(log n)

- **Best Case O(1)**: The key is exactly at the first mid computed (e.g., searching 7 in [1,3,5,7,9,11,13] hits mid=3 on the very first call).
- **Average Case O(log n)**: On average, roughly $\log_2(n)$ comparisons are needed before the key is found or the array is exhausted.
- **Worst Case O(log n)**: Key is at the extreme end or absent; the search splits $\log_2(n)$ times before concluding.
- **Space O(log n)**: Each recursive call adds one frame to the call stack, and there are at most $\log_2(n)$ active calls at once.

6. Paradigm Reflection

6.1 Binary Search is Divide and Conquer

A Divide and Conquer algorithm solves a problem by: (1) **Dividing** it into smaller sub-problems of the same type, (2) **Conquering** each sub-problem recursively, and (3) **Combining** results. Binary search fits perfectly: at each step we divide the array into two halves, conquer only the relevant half (no combination step needed since the answer is a single index). The problem size halves each time, giving O(log n) efficiency.

6.2 Memoized Fibonacci is Dynamic Programming

The core problem with naive Fibonacci is **overlapping subproblems** — fib(k) is recomputed many times for large n. Dynamic Programming (DP) solves this by storing the result of each subproblem the first time it is computed (memoization), so it is never recomputed. This is
top-down DP: we write the natural recursive formula, but add a dictionary cache to remember answers. The result is a drop from $O(2^n)$ to O(n) time — from 2.6 million calls to 59 calls for
n=30.

6.3 Tower of Hanoi Uses Pure Recursion

Base case: When there is only 1 disk, move it directly — no further recursion needed.

Smaller subproblem: To move n disks, we solve the identical problem for n-1 disks (move them to auxiliary), then move the largest disk, then solve n-1 again. Each recursive call reduces n by 1, guaranteeing termination at the base case. Tower of Hanoi is pure recursion — it does not memoize sub-results (unlike DP) nor does it make locally optimal choices (unlike Greedy).

6.4 Real-Life Examples of Algorithm Paradigms

Divide and Conquer — Merge Sort: To sort 1,000 exam papers by roll number, split the pile in half, sort each half separately, then merge the two sorted halves. Each split-and-merge is the same operation at a smaller scale.

Dynamic Programming — GPS Navigation: When a GPS finds the shortest route from city A to city Z, it breaks the journey into sub-routes. The shortest distance to each intermediate city is computed once and stored. Longer routes that reuse that segment look up the stored result rather than recalculate it — classic bottom-up DP (Dijkstra's algorithm).

Greedy — Coin Change (conceptual): To give change for Rs 37 using coins of Rs 20, 10, 5, and 1, a greedy strategy always picks the largest coin that fits: 20 + 10 + 5 + 1 + 1 = 37. At each step we make the locally optimal choice without considering future steps. This works for standard coin denominations but does not always guarantee a globally optimal answer for arbitrary coin sets.